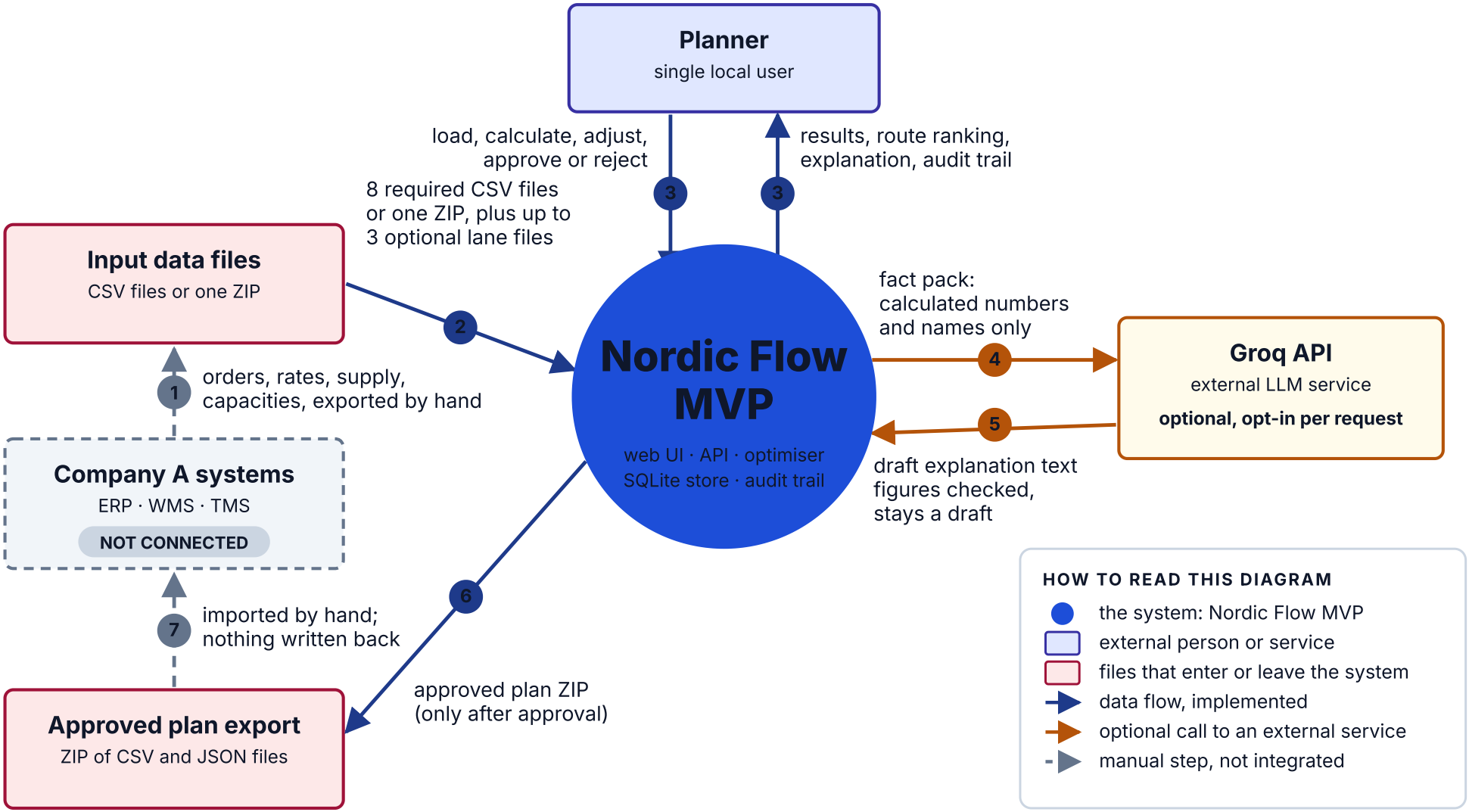
System context: what is inside Nordic Flow and what is around it

Nordic Flow proposes a goods-allocation plan; a planner approves it. Nordic Flow is the circle; every external person, service and file set is a rectangle; each arrow is a labelled data flow, as it works in the MVP. Numbers refer to the register on page 2.



Not in this MVP: live integration with ERP, WMS or TMS · automatic execution or write-back · login and user roles · Groq runs only when a key is set and the planner asks for it

NEW AURA · NORDIC FLOW · ROUND 2

Interactions, trust gates and the life of a plan

Every connection from page 1, what moves across it and how, followed by the checks that stand between a calculation and an exported plan.

Interaction register

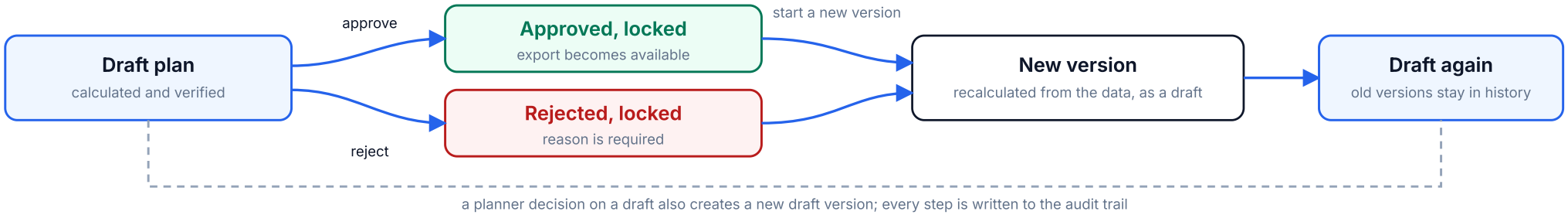
#	FROM → TO	WHAT MOVES	HOW	STATUS
1	**Company A systems → Input files**	Orders, rates, supply and capacities, exported by the planner	Manual CSV export	Manual
2	**Input files → Web UI → API**	8 required CSVs or one ZIP, plus optional lane files; validated before any calculation, errors block the run	HTTP upload, ZIP limits	Implemented
3	**Planner ↔ Web UI**	Load, calculate, review, add a manual route or delivery decision, approve or reject; results, route ranking and audit shown	Browser, localhost:8000	Implemented
4	**Core → Groq API**	Fact pack of numbers and names our code calculated (and product names for the optional SKU matcher). No files or keys	HTTPS JSON, opt-in, key in environment	Optional
5	**Groq API → Core**	Draft explanation; figures are checked against the facts, otherwise it is rejected and a standard summary is used	HTTPS response, stays a draft	Optional
6	**API → Approved plan export**	ZIP of allocations, comparison, planner decisions, manual routes and audit log	HTTP download, only after approval	Implemented
7	**Export → Company A systems**	Planner imports the approved plan; Nordic Flow writes nothing back	Manual import	Manual

Inside Nordic Flow: Web UI → API (JSON over HTTP); API → optimisation core (Python calls) and → SQLite (SQL). The core has no web or database code.

Trust gates

1. **Data validation**
   Eight CSV checks with row-level messages; any error blocks the run.
2. **Independent verifier**
   Re-checks demand, supply, capacity and lanes without using the solver.
3. **Honest saving**
   Shown only against a verified feasible baseline; otherwise the UI says unusable.
4. **AI guardrail**
   The model only rewrites computed facts; a figure not in the facts rejects the text.
5. **Approval gate**
   No export before a planner approves; approved and rejected plans are locked.
6. **Audit trail**
   Append-only events, each hash-linked to the previous one: tamper-evident.

Life of a plan



Synthetic data only; no live link to any Company A system